

## 1 Motivation

Dans l'UE "Philosophie des sciences" en L1, vous avez vu le calcul de Dédution Naturelle (DN) qui est censé représenter le raisonnement humain, contrairement à d'autres calculs logiques, comme la résolution. Un problème de la DN est qu'elle ne se prête pas directement à une automatisation, parce que les règles d'élimination n'ont pas la propriété de sous-formule : lors de l'application d'une règle par chaînage en arrière (réduction de la conclusion d'une règle à ses hypothèses), il faudrait inventer une formule.

Dans ce projet, nous introduisons le calcul des *Tableaux* (à ne pas confondre avec des tables de vérité) qui permet une recherche plus ciblée. Il s'avère que les règles de ce calcul peuvent être vues comme des règles dérivées de la DN, ce qui permet ensuite de convertir une preuve de Tableaux en preuve de DN.

L'objectif de ce projet est donc :

- de se familiariser avec la représentation de formules dans un langage comme Python ;
- d'apprendre à parcourir des formules et de les transformer ;
- de faire connaissance avec les structures de données et algorithmes pour représenter des règles et effectuer une recherche dans un espace d'états ;
- de représenter des dérivations et d'apprendre à transformer des dérivations entre calculs ;
- de faire un usage raisonné d'outils d'Intelligence Artificielle lors de la programmation ;
- de s'initier à de bonnes pratiques de développement de logiciel.

## 2 Structure et déroulement du projet

Le projet est structuré en deux parties :

- Preuves simples avec le calcul des Tableaux : Ces preuves visent à déterminer si un séquent est prouvable ou non, sans effectivement construire une dérivation (c'est à dire, un arbre de preuve).

Cette partie correspond à ce qui est décrit en § 4.1 à § 4.3.1 et § 4.4. A ce point, il suffit d'avoir un prototype qui n'englobe pas toutes les règles du calcul.

A la fin de cette première partie, vous devez rendre votre code, correctement documenté.

- Construction de dérivations dans le calcul des Tableaux, et traduction de ces dérivations vers des dérivations en Dédution Naturelle.

Cette partie correspond à ce qui est décrit en § 4.3.2, § 4.4 à § 4.6. Au delà de cela, quelques extensions et améliorations sont suggérées en § 4.7. Elles sont facultatives et peuvent être effectuées à votre convenance.

A la fin de la deuxième partie, vous devez rendre votre code avec une documentation plus fournie en forme d'un rapport, et présenter votre travail lors d'une soutenance.

### Dates

Pour vous permettre de structurer le semestre, nous indiquons des dates approximatives. Nous lancerons un sondage ultérieurement surtout pour déterminer les dates précises des soutenances :

- Réunion d'information et de discussion (non obligatoire) : autour du 16 octobre 2026
- Rendu première partie : le lundi 2 novembre 2026 à 23h (après les vacances de Toussaint)
- Réunion d'information et de discussion (non obligatoire) : mi-janvier 2027
- Rendu deuxième partie : le lundi 1er février 2027 à 23h
- Soutenances : première semaine de février 2027

### 3 Éléments à rendre

Le travail s'effectue en groupes de 2-3 étudiants (un travail individuel est aussi possible, mais n'est pas conseillé). La programmation s'effectue en Python. Sur demande *préalable*, explicite et bien motivée, d'autres langages de programmation sont acceptables.

Le **code** doit

- ou bien être déposé sur Moodle dans une archive (**tarfile**) avant la date limite. Merci d'utiliser uniquement tar<sup>1</sup> et surtout pas de formats propriétaires. Mettez vous d'accord pour n'effectuer qu'un seul dépôt par groupe. Le dépôt doit contenir un **README** minimaliste avec les noms des membres du groupe.
- ou bien nous envoyer un mail avec un lien vers un dépôt Git ou Mercurial que nous pouvons cloner. Le mail doit être envoyé avant la date limite. Nous nous engageons à ne pas récupérer le dépôt avant la date limite.

Une exigence minimale est que le code soit bien formé et puisse être exécuté. Il est souhaitable qu'il soit correctement documenté. La documentation de haut niveau va dans le rapport, la documentation des fonctions individuelles dans le code. Les deux doivent être complémentaires et pas redondants.

Le **rapport** doit être déposé directement sur Moodle, en format PDF, de préférence écrit en L<sup>A</sup>T<sub>E</sub>X pour ne pas faire subir au lecteur le formatage cruel de formules en Word et Cie. Le rapport doit comporter :

- une courte *introduction* qui décrit le problème sans entrer dans les détails techniques, et qui résume le travail effectué, et peut-être aussi ce que vous n'avez pas réussi à faire ( $\frac{1}{3}$  ou  $\frac{1}{2}$  page) ;
- une petit *manuel d'utilisateur* qui indique comment utiliser votre code ;
- une description de haut niveau de l'implantation.
- le cas échéant des arguments plus théoriques (voir les propositions en § 4.7)

### 4 Indications sur le travail à effectuer

#### 4.1 Représentation et manipulation de formules

L'implantation se fait en Python<sup>2</sup>. Les structures de données vous seront fournies dans un fichier **datastructs.py**. Nous vous conseillons de les utiliser telles quelles sans les modifier, mais vous avez le droit de le faire et devez alors l'indiquer dans votre documentation.

La représentation d'une formule est une instance d'une *classe* de formules, une notion provenant de la programmation orientée objet dont vous pouvez entièrement vous passer pour effectuer le travail demandé.

##### 4.1.1 Construction de formules

Le niveau d'abstraction qu'il convient d'adopter est qu'une **formule** (type **Form**) est construite à partir de constructeurs, selon le principe que vous connaissez du cours Logique 1 (Définition inductive des formules). Une formule est inductivement générée par les constructeurs suivants :

- une constante. La seule constante utilisée est le faux  $\perp$  dont le constructeur est **FF()**.
- une variable propositionnelle. Le constructeur prend comme argument le nom de la variable (une chaîne de caractères), par exemple **Var("a")**.
- une négation d'une formule  $f$ . Le constructeur **Not** prend un argument, la formule  $f$ , donc **Not(f)** pour  $\neg f$ .
- un opérateur binaire entre deux formules  $f$  et  $g$ . Les constructeurs s'écrivent comme applications de fonctions préfixes, donc **And(f,g)** pour  $f \wedge g$  ; **Or(f,g)** pour  $f \vee g$  ; **Impl(f,g)** pour  $f \rightarrow g$ .

---

1. <https://www.gnu.org/software/tar/manual/tar.html>

2. D'autres langages sont acceptables, mais il faut impérativement nous en parler au début du projet.

Par exemple, la formule  $(a \wedge \perp) \rightarrow \neg b$  est codée par `Impl(And(Var("a"), FF()), Not(Var("b")))`.

Une erreur fréquente est d'oublier le constructeur `Var`, qui est essentiel pour distinguer une variable propositionnelle (de la formule) d'une variable (du langage de programmation) qui représente une formule. `Impl(And(a, FF()), Not(b))` donne une erreur, sauf si vous avez affecté des formules à `a` et `b` avant.

Vous pouvez afficher une formule sous une forme plus lisible avec `print`.

```
>>> f = Impl(And(Var("a"), FF()), Not(Var("b")))
>>> f                                     # la représentation brute des formules
Impl(left=And(left=Var(name='a'), right=FF()), right=Not(form=Var(name='b')))
>>> print(f)                             # l'affichage de formules
((a ∧ ⊥) → ¬ (b))
```

#### 4.1.2 Déconstruction de formules

Il y a deux manières alternatives de décomposer une formule.

**Avec des accesseurs** Étant donnée une formule, vous pouvez accéder à ses constituants ou sous-formules via des *accesseurs*, qui sont les champs que vous trouvez dans la définition de classe des formules. Ainsi, `name` permet de récupérer le nom d'une variable (de type `str`), et `left` et `right` sont les accesseurs pour les sous-formules de formules binaires. Les accesseurs peuvent être imbriqués :

```
>>> g = Impl(Impl(Var("a"), FF()), Not(Var("b")))
>>> g.left.right
FF()
```

Evidemment, les accesseurs `left` et `right` ne sont pas définis pour tous les constructeurs. Pour tester si une formule est une instance d'une application de constructeur, utiliser `isinstance` :

```
>>> isinstance(g, Impl)
True
>>> isinstance(g, Not)
False
```

**Par filtrage** Le filtrage<sup>3</sup> s'avère utile s'il faut appliquer un traitement à une formule, cas par cas, selon sa structure. L'expression suivant le `match` est successivement examinée par tous les `case`, et le premier *motif* approprié est sélectionné et les sous-expressions sont affectées aux variables du motif, pour exécuter le code du `case`. Voici une fonction pour éliminer des implications d'une formule :

```
def elimImpl(f: Form) -> Form:
    match f:
        case Impl(a,b):
            return Or(Not(elimImpl(a)), elimImpl(b))
        case And(a, b):
            return And(elimImpl(a), elimImpl(b))
        case f:
            return f
```

---

3. voir <https://peps.python.org/pep-0636/> pour des détails

Le premier cas correspond donc à une équation récursive

```
elimImpl(Impl(a,b)) = Or(Not(elimImpl(a)), elimImpl(b))
```

telle que vous la connaissez du cours Logique 1. Ici, **a** et **b** sont des variables qui représentent des formules et pas des variables propositionnelles. Notez que la fonction est incomplète (quelques cas ne sont pas traités correctement) - complétez-la !

Voici le fonctionnement :

```
>>> print(g)
((a → ⊥) → ¬(b))
>>> print(elimImpl(g))
(¬((¬(a) ∨ ⊥)) ∨ ¬(b))
```

Dans la définition, le dernier **case f** est toujours applicable et renvoie la formule inchangée.

Python ne vous avertit pas qu'un filtrage est incomplet ; l'exécution continue silencieusement avec la construction de formules mutilées. Pour cela, il est utile de toujours terminer un **match ...case** avec un cas *catch-all* de cette forme (ici **case f**).

Si un cas ne devrait pas apparaître, levez une exception, avec **raise** et un message d'erreur. Cela arrête le programme immédiatement.

---

```
case f:
    raise ValueError("impossible_d'avoir_ce_motif_ici")
```

---

Si une variable n'est pas utilisée dans un motif, on peut la remplacer par un underscore :

---

```
case _:
    raise ValueError("impossible_d'avoir_ce_motif_ici")
```

---

Des motifs peuvent aussi être imbriqués ; ici pour supprimer une double négation :

---

```
case Not(Not(f)):
    return f
```

---

Les deux mécanismes de déconstruction (accesseurs et filtrage) peuvent aussi être utilisés pour les autres structures introduites dans la suite (séquents, dérivations, ...).

#### A faire

Essayez de vous familiariser avec la structure des formules. Ecrivez quelques fonctions sur les formules (compter le nombre de feuilles d'une formule, renvoyer la liste des variables d'une formule, mettre une formule en forme normale négative). Ces fonctions ne sont pas très utiles dans la suite, mais vous voyez comment traverser (récursivement) une formule.

## 4.2 Codage de règles

Vous vous souvenez de la définition d'un *séquent*  $h_1, \dots, h_n \vdash c$  : une liste d'hypothèses  $h_1, \dots, h_n$  (des formules) et une conclusion  $c$  (une formule). Le type Python correspondant est **Seq**.

Vous vous souvenez aussi des *règles*. Un ensemble de règles donne un *calcul*. Nous résumons les règles de DN (que vous connaissez) en § A.1 et les nouvelles règles du calcul des Tableaux (calcul T) en § A.2, avec quelques explications supplémentaires dans § A.3.

Dans notre cas, une règle a une signification *opérationnelle* : elle permet de transformer un séquent (la conclusion d'une règle), en d'autres séquents (zéro, un ou deux dans le cas du calcul T), les prémisses de la règle. Cela correspond à la recherche de preuves par chaînage en arrière déjà pratiquée, de manière

artisanale, pour le calcul DN. Comme déjà constaté, DN est inadapté pour la recherche de preuves systématique, et nous nous concentrons sur le calcul T.

L'implantation d'une règle est donc composée :

- d'un nom (pour la lisibilité)
- d'une condition d'applicabilité (pour vérifier que la règle est applicable : ce n'est pas la dérivabilité d'un séquent)
- d'une action qui génère de nouveaux séquents

Déclinons une **première version** de ces concepts. Prenons l'exemple des règles :

$$\frac{\Gamma, \neg A \vdash \perp}{\Gamma \vdash A} (S) \quad \frac{A \wedge B \in \Gamma \quad \Gamma, A, B \vdash \perp}{\Gamma \vdash \perp} (\alpha\wedge)$$

(S) est toujours applicable, la condition d'applicabilité est donc **True**. L'action est de convertir le séquent  $\Gamma \vdash A$  en  $\Gamma, \neg A \vdash \perp$

( $\alpha\wedge$ ) est applicable sous condition que  $A \wedge B \in \Gamma$ . L'action est de convertir le séquent  $\Gamma \vdash \perp$  en  $\Gamma, A, B \vdash \perp$

Dans un contexte d'opérationnalisation des règles, une autre considération s'impose rapidement : comment assurer la terminaison de la recherche avec un calcul ? On voit que dans sa forme actuelle, (S) est toujours applicable au séquent  $\Gamma \vdash \perp$  et ses descendants ; et pareil pour ( $\alpha\wedge$ ) et un séquent comme  $p \wedge q \vdash \perp$ . Une **deuxième version** introduit donc des conditions supplémentaires qui empêchent l'application d'une règle si cette application "n'apporterait rien de nouveau".<sup>4</sup> Voici donc les règles avec des conditions d'arrêt :

$$\frac{A \neq \perp \quad \Gamma, \neg A \vdash \perp}{\Gamma \vdash A} (S) \quad \frac{A \wedge B \in \Gamma \quad (A \notin \Gamma \text{ ou } B \notin \Gamma) \quad \Gamma, A, B \vdash \perp}{\Gamma \vdash \perp} (\alpha\wedge)$$

Vous voyez que (S) n'est plus applicable à un séquent de la forme  $\Gamma \vdash \perp$ , et ( $\alpha\wedge$ ) n'est plus applicable si  $A$  et  $B$  sont tous les deux déjà dans  $\Gamma$ . Soyez sûr de comprendre que ces transformations des règles n'impactent pas la correction et complétude du calcul, voir aussi § 4.7.1.

Dans une **troisième version**, nous procédons au codage des règles en Python. C'est cette version que nous vous demandons d'implanter.

- Une condition d'applicabilité est une fonction qui prend une liste de formules **fs** (les cibles de la règle) et un séquent **s** et renvoie un booléen (voir type **Appcond**). C'est à vous de préciser ce que vous entendez par "cibles de la règle" ; une interprétation raisonnable est de dire que ces formules **fs** sont des formules appartenant aux antécédents du séquent **s** et qui sont décomposées par la règle (pour ( $N$ ),  $\alpha$  ou  $\beta$ ) ou autrement manipulées par la règle (pour ( $AxC$ )). Pour la règle (S) qui manipule le succédent d'un séquent ou pour ( $AxF$ ), la liste **fs** est vide.
- Une action a les mêmes paramètres qu'une condition d'applicabilité, mais renvoie une liste de séquents (voir type **Action**).

Prenons les deux exemples. Pour la règle (S) :

---

```
# Condition d'applicabilite
def s_t_appcond (fs: list[Form], s: Seq) -> bool:
    match (fs, s):
        case ([], Seq(_, a)):
            return (a != FF())
        case _:
            return(False)

# Action
```

---

4. Heureusement, il suffit de regarder le séquent localement sans prendre en compte toute la dérivation qui mène à lui.

```
# définissez la fct insert_forms qui évite des doublons
def s_t_action (fs: list[Form], s: Seq) -> list[Seq]:
    match (fs, s):
        case ([], Seq(g,a)):
            return ([Seq(insert_forms (g, [Not(a)]), FF())])
        case _:
            return ([])

# Règle
s_t_rule = Rule ("s_t", s_t_appcond, s_t_action)
```

---

Pour la règle  $(\alpha\wedge)$ , nous codons uniquement la condition :

---

```
def a_and_t_appcond (fs: list[Form], s: Seq) -> bool:
    match (fs, s):
        case ([And(a,b)], Seq(g,False)):
            return (not_both_in (a, b, g))
        case _:
            return (False)
```

---

A vous de coder `not_both_in(a, b, g)`, qui correspond à  $A \notin \Gamma$  ou  $B \notin \Gamma$ . Notez qu'un invariant des règles est que les formules `fs` sont dans l'antécédent de `s`. Pour cela, on ne teste pas explicitement `And(a,b) in g`.

#### A faire

Codez les règles du calcul des Tableaux (la troisième version). Prenez juste quelques règles et codez-les. Dans votre rapport, justifiez votre choix des conditions d'applicabilité. Complétez les règles après avoir réussi à implanter la recherche de preuves en § 4.4.

### 4.3 Application de règles

Après définition des règles, nous nous intéressons maintenant à l'application d'une règle à un *état de preuve* dans le cadre d'une recherche de preuves.

Nous procédons en deux étapes. Nous appliquerons d'abord une méthode basique, qui correspond aussi à une méthode de recherche basique. Une fois cette méthode est opérationnelle, nous enrichissons les structures de données, ce qui nécessite un raffinement de l'application des règles, qui est la base d'un raffinement de la méthode de recherche. C'est uniquement avec cette méthode plus poussée que nous aurons la possibilité de construire des dérivations.

#### 4.3.1 Application basique d'une règle

**Notion d'état de preuve** Une notion centrale pour l'application d'une règle est un *état de preuve* (proof state). Dans sa version basique, c'est une liste de séquents, à savoir les séquents qu'il faut encore prouver.

---

```
ProofStateV0 = list[Seq]
```

---

Pour illustrer le concept, considérons la preuve partielle :

$$\frac{\neg p \vee q, \neg(p \wedge q), \neg\neg p, \neg p \vdash \perp \quad \neg p \vee q, \neg(p \wedge q), \neg\neg p, q \vdash \perp}{\neg p \vee q, \neg(p \wedge q), \neg\neg p \vdash \perp} (\beta\vee)$$

L'état de preuve est la liste des séquents ouverts :  $[(\neg p \vee q, \neg(p \wedge q), \neg\neg p, \neg p \vdash \perp), (\neg p \vee q, \neg(p \wedge q), \neg\neg p, q \vdash \perp)]$ . Pour l'instant, nous oublions le séquent dont ils sont issus, à savoir  $\neg p \vee q, \neg(p \wedge q), \neg\neg p \vdash \perp$ . Ce dernier sera pris en compte en § 4.3.2.

Pour faire avancer cet état de preuve, nous avons deux degrés de liberté :

- Choix du séquent : dans l'exemple, le premier ou le deuxième.
- Dans le séquent sélectionné, choix de la règle à appliquer et position / formule à laquelle elle est appliquée.

Les choix qu'on adopte ici déterminent la *stratégie de preuve*, et nous vous demanderons à mener une réflexion à ce sujet dans § 4.7.3. Pour l'instant, nous faisons simple.

**Choix du séquent** Le choix du séquent est en général plutôt<sup>5</sup> indifférent : on n'échappera pas à prouver tous les séquents ouverts. On peut donc prendre le premier but, ce que nous faisons ici.

**Choix des règles et positions** Une fois le séquent sélectionné, nous le parcourons pour identifier les formules auxquelles on peut appliquer une règle. Pour le séquent  $s = \neg p \vee q, \neg(p \wedge q), \neg\neg p, \neg p \vdash \perp$ , regardons :

- la règle (*S*) n'est pas applicable au succédent de  $s$
- aux formules  $fs = [\neg p \vee q]$  de l'antécédent, la règle ( $\beta\vee$ ) n'est plus applicable
- aux formules  $fs = [\neg(p \wedge q)]$ , la règle ( $\beta\wedge$ ) n'est pas applicable.
- aux formules  $fs = [\neg\neg p]$ , la règle (*N*) est applicable et produit les nouveaux séquents  $[\neg p \vee q, \neg(p \wedge q), \neg\neg p, \neg p \vdash \perp]$  (donc en fait un nouveau séquent)
- aux formules  $fs = [\neg p]$ , la règle (*AxC*) est applicable et produit les nouveaux séquents  $[]$ .

Il y a plusieurs façons d'organiser ce processus :

- parcourir toutes les règles, et pour chaque règle, identifier les formules du séquent auxquelles elle est applicable, et l'appliquer ; *ou*
- parcourir toutes les formules du séquent et trouver les règles qui soient applicables, et les appliquer.

A vous de choisir.

**Fonction next\_proof\_state** L'important est que vous arrivez à définir une fonction

---

```
def next_proof_state (ps: ProofStateV0) -> list[ProofStateV0]:
```

---

qui prend un état de preuve et génère une liste d'états de preuves.

Cette fonction dépend d'un calcul (à savoir une liste de règles), que vous pouvez fixer comme variable globale. Nous conseillons de commencer avec un petit calcul qui ne contient que quelques règles : `[s.t.rule, a.and.t.rule, ...]` pour l'étendre au calcul T complet plus tard.

La fonction `next_proof_state` prend donc le premier séquent de l'état de preuve<sup>6</sup> et pour toute règle / formule du séquent, le remplace par la liste des séquents successeurs.

Revenons à notre exemple. L'état de preuve est :  $[(\neg p \vee q, \neg(p \wedge q), \neg\neg p, \neg p \vdash \perp), (\neg p \vee q, \neg(p \wedge q), \neg\neg p, q \vdash \perp)]$  La fonction `next_proof_state` produit deux nouveaux états de preuve :

- $[(\neg p \vee q, \neg(p \wedge q), \neg\neg p, \neg p, p \vdash \perp), (\neg p \vee q, \neg(p \wedge q), \neg\neg p, q \vdash \perp)]$ . Le premier séquent a été remplacé par un nouveau séquent par application de la règle (*N*) à  $[\neg\neg p]$ .
- $[(\neg p \vee q, \neg(p \wedge q), \neg\neg p, q \vdash \perp)]$ . Le premier séquent a été supprimé par application de la règle (*AxC*) à  $[\neg p]$ .

#### A faire

Définissez la fonction `next_proof_state`. C'est la seule fonction algorithmiquement "difficile" parce qu'elle itère sur toutes les règles d'un calcul et toutes les positions d'un séquent.

5. Ceci n'est pas entièrement vrai, surtout si quelques séquents ne sont pas prouvables, voir § 4.7.2

6. La liste des séquents sera toujours non-vide, voir § 4.4

Nous manipulons ici différentes formes de listes, dont le statut est différent :

- un état de preuve contient une *liste de séquents*, et, à la fin, l'objectif est d'avoir prouvé *tous* les séquents qu'il contient ;
- la fonction `next_proof_state` génère une *liste d'états de preuve*, qui correspondent à des alternatives de construction d'une preuve. Ici, il suffit qu'*il existe* un seul état de preuve qui soit prouvable. Les autres correspondent peut-être à une fausse route suite à la mauvaise application d'une règle. Il faut pourtant se poser si générer toutes les alternatives est utile – une réflexion que vous pourrez approfondir dans § 4.7.4.

Nous pouvons maintenant aborder la recherche de preuves. **Passez donc d'abord à la § 4.4 avant de revenir à la section suivante pour raffiner l'application des règles.**

#### 4.3.2 Raffinement d'application d'une règle

Nous introduisons un raffinement du type d'état de preuve qui permet de mémoriser comment l'état de preuve évolue. Cela est nécessaire afin de reconstruire une dérivation, c'est à dire un arbre de preuve. Le problème est que la définition typique, inductive, d'un arbre (voir la définition en § 4.5) permet de faire croître un arbre à la racine (en rassemblant des sous-arbres) mais pas aux feuilles (ce qui se fait dans une preuve).

Pour contourner le problème, nous introduisons deux mesures :

- Nous ne supprimons pas les séquents devenus inactifs après application d'une règle, mais nous les mémorisons dans une structure de "but fermé" (`ClosedGoal`). Nous gardons aussi les séquents actifs et les appelons `OpenGoal`.
- Pour savoir quel séquent a engendré quel(s) séquent(s), nous gardons un historique (`history`) qui code la position de chaque séquent dans la dérivation.

Voici la définition de ces structures et du `ProofState` définitif :

---

```
class OpenGoal:
    history: History
    seq: Seq
class ClosedGoal:
    history: History
    seq: Seq
    rulename: Rulename
    forms: list[Form]
ProofState = tuple[list[OpenGoal], list[ClosedGoal]]
```

---

L'historique est une séquence de nombres. La racine de l'arbre a la liste vide `[]` comme historique. Pour chaque noeud de l'arbre avec historique `h`, ses  $n$  fils ont l'historique `h + [0] .. h + [n - 1]`.

Prenons l'arbre de preuve en § A.3 comme exemple. La racine a l'historique `[]`, son fils l'historique `[0]`, le fils de celui-ci `[0, 0]`, et les deux feuilles respectivement `[0, 0, 0]` et `[0, 0, 1]`.

Un état de preuve est maintenant un couple de listes (buts ouverts, buts fermés). Tant qu'il y a des buts ouverts, prenez-en le premier, appliquez une règle comme dans la version basique de § 4.3.1, et annotez les séquents ainsi obtenus avec des historiques. Ce seront de nouveaux buts stockés dans le premier composant du `ProofState`. Le but ouvert actuel est converti en but fermé. Il garde les composants `history` et `seq`. Dans le deuxième composant du `ProofState`, on mémorise la règle appliquée (`rulename`) ainsi que les formules associées `forms` (voir les `Appcond` et `Action` des règles).

#### A faire

Étendez la version basique de `next_proof_state`, qui est maintenant de type `ProofState` -> `list[ProofState]`. Adaptez ensuite les fonctions `sol_proof_state` et `init_state` de § 4.4.

## 4.4 Recherche de preuves

La recherche de preuves repose sur un algorithme, **dfs** (*depth first search*), que nous adoptons ici. Cet algorithme explore une liste d'états (**states**). Dans notre cas, ce sont les états de preuve de § 4.3. L'algorithme prend en plus deux opérateurs :

- **nexts**, une fonction qui prend un état et produit une liste d'états. C'est justement la fonction **next\_proof\_state** de § 4.3.
- **sol**, une fonction qui prend un état et teste s'il est une solution ou non. Elle renvoie un booléen. Cette fonction reste à programmer, (voir la fonction **sol\_proof\_state** plus bas).

---

```
PS = TypeVar("PS")
def dfs (nexts: Callable[[PS], list[PS]],
        sol: Callable[[PS], bool],
        states: list[PS]) -> list[PS]:
    if states == []:
        return ([])
    else:
        if sol(states[0]):
            return [states[0]]
            # si vous souhaitez continuer la recherche, décommentez:
            # return [states[0]] + dfs(nexts, sol, nexts(states[0]) + states[1:])
        else:
            return dfs(nexts, sol, nexts(states[0]) + states[1:])
```

---

Le but de la fonction est de renvoyer une liste de solutions pour un problème de recherche. Si la liste des états est vide, il n'y a pas de solution, elle renvoie donc la liste vide. Si le premier état est une solution (selon la fonction **sol**), on le renvoie. Sinon, on génère les états successeurs du premier état avec **nexts** et les place devant les états existants, et on continue la recherche. Au lieu d'arrêter la recherche après la première solution trouvée, on peut aussi la continuer pour une recherche exhaustive, en effectuant un autre appel récursif.

**Digression** La fonction est entièrement générique, elle peut donc être utilisée pour des “états de preuve” de différentes formes. Un “état” peut aussi être un nombre naturel, et nous pouvons utiliser **dfs** dans une recherche bizarroïde pour trouver des nombres naturels < 40 divisibles par 5. Définissez :

---

```
def nexts_nat (n: int) -> int:
    if n < 20:
        return [2 * n, 2 * n + 1]
    else:
        return []
def sol_nat (n: int) -> bool:
    return (n % 5 == 0)
```

---

et lancez **dfs(nexts\_nat, sol\_nat, [1])**. Expérimentez avec les deux variantes (une seule solution ou recherche exhaustive). Un **print(states)** au début de la fonction **dfs** permet de comprendre pourquoi la fonction produit une séquence de solutions “désordonnées” à première vue.

Revenons aux preuves.

### A faire

Définissez la fonction **sol\_proof\_state** qui prend un état de preuve et vérifie qu'il est une solution. Cette fonction est très simple :

- Dans la version basique (type **ProofStateV0**), nous avons une solution quand la liste des

séquents à prouver est vide. Cette version est à fournir dans la première étape du projet.

- Dans la version raffinée (**ProofState**), une solution est un état sans but ouvert. Cette version est à fournir dans la deuxième étape du projet.

Définissez aussi une fonction **init\_state** qui prend un séquent et crée un état initial.

Vous pouvez maintenant définir

```
def prove_seq (sq: Seq) -> list[ProofState]:  
    dfs(next_proof_state, sol_proof_state, init_state(sq))
```

et la tester pour la preuve de séquents :

- Dans la version basique, les résultats sont peu spectaculaires : Si un séquent n'est pas prouvable, la fonction de preuve renvoie une liste vide ; sinon, elle renvoie une liste avec un état solution (donc une liste de liste vide). Après quelques tests, revenez à § 4.3.2 pour raffiner la notion d'état de preuve.
- Dans la version raffinée, un état solution est plus informatif, et nous allons en profiter pour construire un arbre de dérivation.

#### Fin de première étape

La fin de cette section marque la fin de la première étape du projet. La deuxième étape du projet aborde les sections suivantes, et nécessite de revoir la notion d'état de preuve, voir § 4.3.2.

## 4.5 Construction de l'arbre de dérivation

La définition inductive de "dérivation" est difficile à cause de sa simplicité : une dérivation contient (essentiellement) un séquent et une liste de dérivations.

La plupart des définitions inductives de structures arborescentes que vous avez vues jusqu'à maintenant ont des constructeurs de base (non-inductifs comme **Var** pour les formules) et des constructeurs inductifs (comme **Not** et **And**). Pour les dérivations, il n'y a qu'un seul constructeur, qui est inductif : c'est l'application d'une règle d'inférence, qui génère d'autres dérivations (correspondant aux prémisses de la règle). Le cas de base est un peu caché : c'est une dérivation dont la liste des dérivations (des prémisses) est vide. Voici donc la définition du type :

```
class Deriv:  
    seq: Seq  
    rulename: Rulename  
    forms: list[Form]  
    children: list["Deriv"]
```

Les composants **seq**, **rulename** et **forms** sont exactement comme pour **ClosedGoal**.

Regardons une dérivation très simple, du séquent  $a, \neg a \vdash \perp$  :

```
[Deriv (  
    Seq ([Var("a"), Not (Var("a"))], FF()),  
    "axc_t",  
    [Var("a")],  
    [])]
```

Le séquent a été prouvé en appliquant ( $AxC$ ) à la formule  $a$ , avec une sous-dérivation vide. Et voici une dérivation un peu plus complexe, de  $a \vee b, \neg a, \neg b \vdash \perp$ , qui contient (des variantes de) la première comme sous-dérivations :

---

```
[Deriv (
  Seq ([Or (Var("a"), Var("b")), Not (Var("a")), Not (Var("b"))], FF()),
  "b_or_t",
  [Or (Var("a"), Var("b"))],
  [Deriv(
    Seq ([Var("b"), Or (Var("a"), Var("b")), Not (Var("a")), Not (Var("b"))], FF()),
    "axc_t",
    [Var("b")],
    []),
  Deriv(
    Seq ([Var("a"), Or (Var("a"), Var("b")), Not (Var("a")), Not (Var("b"))], FF()),
    "axc_t",
    [Var("a")],
    [])]]]
```

---

Comment construire une dérivation à partir du résultat de preuve de la § 4.4? Nous observons que toute solution d'une preuve est uniquement composée de buts fermés.

La démarche est en principe simple mais peut-être insolite :

- nous identifions le but (de type **ClosedGoal**) dont l'historique est []. Ce sera le but actuel.
- Nous construisons récursivement la liste des dérivations pour les fils du but actuel.
- Nous construisons **Deriv(...)** contenant l'information du but actuel et la liste des dérivations des fils.

En détail :

- Vous avez besoin d'une fonction

---

```
def is_child (child_hist: ClosedGoal, curr_hist: ClosedGoal) -> bool:
```

---

qui dit qu'un historique est le fils d'un autre historique, par exemple **is\_child**([0,1,1], [0,1]) mais pas **is\_child**([0,1,1], [0,0]). *Note* : avec **xs[:-1]**, vous récupérez le reste d'une liste **xs**. Attention aux listes vides!

- La fonction

---

```
def derivation_tree (gls: list[ClosedGoal], curr_goal: ClosedGoal) -> Deriv:
```

---

fait l'essentiel du travail (mais on peut l'écrire en 3 lignes). Ici, **gls** est la liste de tous les buts, **curr\_goal** le but actuel. Si vous utilisez des compréhensions de listes, voir § B.2, tout est très facile à écrire.

### A faire

Écrivez les fonctions mentionnées dans cette section.

Comme bonus, vous pouvez aussi écrire des fonctions qui affichent des dérivations dans un format plus lisible, ou bien en affichant des séquents indentés en fonction de leur profondeur dans la dérivation; ou même en  $\text{\LaTeX}$ <sup>a</sup>.

---

a. voir le paquetage <https://ctan.org/pkg/lkproof>

## 4.6 Transformation de l'arbre de dérivation

La dernière étape consiste à transformer une preuve en calcul T vers une preuve en DN.

Il s'avère que les règles du calcul T peuvent être comprises comme des règles dérivées de DN. Regardons la règle (*N*) du calcul T. Supposez que vous avez fait une preuve en calcul T :

$$\frac{\frac{\mathcal{D}}{\neg\neg A \in \Gamma} \quad \frac{}{\Gamma, A \vdash \perp}}{\Gamma \vdash \perp} (N)$$

La preuve correspondante en DN est :

$$\frac{\frac{\neg\neg A \in \Gamma}{\Gamma \vdash \neg\neg A} (Ax) \quad \frac{\frac{\mathcal{D}^T}{\Gamma, A \vdash \perp} (I\neg) \quad \frac{}{\Gamma \vdash \neg A} (E\neg)}{\Gamma \vdash \perp}$$

où  $\mathcal{D}^T$  est la dérivation  $\mathcal{D}$  transformée récursivement.

Écrivez donc la fonction

---

```
def translate_derivation (d: Deriv) -> Deriv:
```

---

qui effectue la traduction. Cette fonction s'écrit en deux lignes (utilisez encore une fois la compréhension de listes de § B.2 pour appliquer **translate\_derivation** récursivement à ses sous-listes), pourvu que nous ayons une définition

---

```
def translate_rule (rn: Rulename, sq: Seq, fms: list[Form], ds: list[Deriv]) -> Deriv:
```

---

qui traduit chaque règle du calcul T (entre autres (N)). Le plus facile est d'écrire **translate\_rule** comme une fonction qui cherche une dérivation (paramétrée) pour chaque nom de règle dans un dictionnaire.

Ainsi, nous aurions par exemple la fonction **deriv\_rule\_n** qui a (presque) le même profil que **translate\_rule**, avec le nom de la règle en moins, et qui code la dérivation :

---

```
def deriv_rule_n (sq: Seq, fms: list[Form], ds: list[Deriv]) -> Deriv:
    match (sq, fms, ds):
        case (Seq(g, FF()), [Not(Not(a))], [d]):
            Deriv (Seq(g, FF()), "e_not_nd", [Not(a)], \
                [ \
                    Deriv (Seq(g, Not(Not(a))), "ax_nd", [], []), \
                    Deriv (Seq(g, Not(a)), "i_not_nd", [], [d]) \
                ] \
            )
```

---

Les dérivations **ds** sont les dérivations déjà traduites récursivement qui sont greffées aux endroits appropriés de l'expansion de la règle.

*Note (Python)* : une expression peut s'étendre sur plusieurs lignes si les lignes sont suivies d'un backslash \.

#### A faire

Complétez la traduction pour les autres règles. Vous avez le droit de ne pas traduire exhaustivement toutes les règles, mais de pouvoir illustrer le principe sur des cas de test crédibles.

Vous constaterez que les preuves deviennent vite très grosses et illisibles, et elles sont peu "naturelles". L'extension en § 4.7.4 propose d'y remédier.

## 4.7 Extensions

### Travail facultatif

Les extensions proposées dans cette section sont facultatives. Vous pouvez les aborder (ou non) selon vos intérêts personnels. Elles vous donnent un bonus dans l'évaluation mais ne sont pas un prérequis même pour une très bonne note.

Nous vous invitons à réfléchir à des extensions du programme de base décrit jusqu'à maintenant. Les sections suivantes donnent quelques pistes.

### 4.7.1 Correction et complétude du calcul T

Ceci est un travail de réflexion plutôt que de programmation, où il s'agit d'argumenter que l'algorithme des Tableaux est correct et complet.

Vous vous souvenez de la relation de conséquence sémantique  $H_1, \dots, H_n \models C$  satisfaite si et seulement si tout modèle de  $H_1, \dots, H_n$  satisfait  $C$ , et ceci est le cas si et seulement si  $\{H_1, \dots, H_n, \neg C\}$  a un modèle. Motivé par cette équivalence, nous allons dire qu'un séquent  $H_1, \dots, H_n \vdash C$  est un modèle de l'ensemble  $\{H_1, \dots, H_n, \neg C\}$ .

Veuillez élaborer les points suivants :

**Correction** Les prémisses des règles ont les mêmes modèles que la conclusion de la règle (précisez ce que cela veut dire si une règle a plusieurs prémisses).

*Exemple* : Règle  $(\alpha \wedge)$  : Sous condition que  $A \wedge B \in \Gamma : \Gamma$  et  $\Gamma, A, B$  ont les mêmes modèles.

**Complétude** Nous disons qu'un séquent  $\Gamma \vdash C$  est entièrement décomposé si l'application des règles  $(S)$ ,  $(N)$ ,  $\alpha$  et  $\beta$  ne modifie pas  $C$  et ne rajoute plus de nouvelles formules à  $\Gamma$  (notre critère d'arrêt en § 4.2). Montrez que si un séquent est entièrement décomposé :

- ou bien une des règles  $(Ax F)$  ou  $(Ax C)$  est applicable. On dit que le séquent est *fermé*.
- ou bien le séquent a un modèle. On dit que le séquent est *ouvert*.

Notre calcul vise à montrer que *tous les séquents sont fermés*, ce qui est interprété comme succès de la recherche d'une preuve (et nous pourrions construire une dérivation qui valide le séquent original).

On peut aussi comprendre la méthode des tableaux comme une recherche de modèles. Sous cette optique, trouver *un seul séquent ouvert* est le but recherché. Ce résultat permet de construire un contre-modèle du séquent original.

*Exemple* : Prenons le séquent  $p \wedge q \vdash p \longrightarrow r$ . En appliquant les règles du calcul T, nous arrivons au séquent entièrement décomposé  $p \wedge q, \neg(p \longrightarrow r), p, q, \neg r \vdash \perp$ . Un contre-modèle de ce séquent est la valuation  $v(p) = 1, v(q) = 1, v(r) = 0$ .

### 4.7.2 Construction de contre-exemples

Dans sa version basique, l'algorithme produit une liste de dérivations vide si un séquent n'est pas prouvable. Suite à la réflexion de § 4.7.1, vous pouvez modifier l'algorithme de recherche pour qu'il renvoie ou bien une preuve d'un séquent, ou bien un contre-modèle. Pour le faire, il suffit de

- modifier l'algorithme de recherche de solution, de manière minimaliste : il faut que la fonction **proofSolution** accepte non seulement un but fermé, mais aussi un but ouvert si le séquent est entièrement décomposé.
- différencier le traitement ultérieur : les buts fermés (dans le sens utilisé jusqu'à maintenant) sont transformés en dérivation comme avant ; des séquents entièrement décomposés et ouverts sont convertis en contre-modèle.

### 4.7.3 Optimisations de la recherche dans le calcul T

Réfléchissez à optimiser votre code avec un effort raisonnable.

- Sur un plan conceptuel : ordre d’application des règles. Privilégier l’application de certaines règles du calcul T et retarder l’application d’autres peut éviter de dupliquer le travail.
- Sur un plan de l’implantation : comment identifier au mieux les règles applicables à un séquent ?

De toute façon, vos optimisations ne devraient pas se faire au détriment de la clarté ou de la robustesse de votre code. Ainsi, évitez des spécificités de représentation de structures de pointeurs en Python, ou des opérations avec effet de bord difficilement maîtrisables.

### 4.7.4 Optimisations dans la construction de la dérivation en DN

Vous constatez que les dérivations en Dédution Naturelle obtenues par traduction des dérivations du calcul T sont correctes mais pas du tout naturelles. Elles passent le plus souvent par une utilisation de la règle du tiers-exclus. Essayez de

- rajouter des règles au calcul T (par exemple les règles d’introduction de la DN).
- modifier des règles du calcul T. Par exemple, la règle  $(\alpha \wedge)$  reste correcte si le succédent  $\perp$  est remplacé par n’importe quelle autre formule. Quelle serait l’impact de cette modification sur le calcul ?

Toutes ces mesures rendent la recherche plus non-déterministe, et certaines règles ne sont pas “inversibles” (par exemple les règles  $(I\vee)$  de la DN). Elles induisent donc la possibilité de faire “fausse route” et nécessitent d’explorer des alternatives – et c’est uniquement à ce stade que les méthodes de recherche de la § 4.4 deviennent vraiment intéressantes.

## A Règles

### A.1 Dédution naturelle : Calcul DN

|               | Introduction                                                                                                              | Élimination                                                                                                                       |
|---------------|---------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| $\wedge$      | $\frac{\Gamma \vdash A \quad \Gamma \vdash B}{\Gamma \vdash A \wedge B} (I\wedge)$                                        | $\frac{\Gamma \vdash A \wedge B}{\Gamma \vdash A} (E\wedge_1) \quad \frac{\Gamma \vdash A \wedge B}{\Gamma \vdash B} (E\wedge_2)$ |
| $\rightarrow$ | $\frac{\Gamma, A \vdash B}{\Gamma \vdash A \rightarrow B} (I\rightarrow)$                                                 | $\frac{\Gamma \vdash A \rightarrow B \quad \Gamma \vdash A}{\Gamma \vdash B} (E\rightarrow)$                                      |
| $\neg$        | $\frac{\Gamma, A \vdash \perp}{\Gamma \vdash \neg A} (I\neg)$                                                             | $\frac{\Gamma \vdash \neg A \quad \Gamma \vdash A}{\Gamma \vdash \perp} (E\neg)$                                                  |
| $\perp$       |                                                                                                                           | $\frac{\Gamma \vdash \perp}{\Gamma \vdash A} (E\perp)$                                                                            |
| $\vee$        | $\frac{\Gamma \vdash A}{\Gamma \vdash A \vee B} (I\vee_1) \quad \frac{\Gamma \vdash B}{\Gamma \vdash A \vee B} (I\vee_2)$ | $\frac{\Gamma \vdash A \vee B \quad \Gamma, A \vdash C \quad \Gamma, B \vdash C}{\Gamma \vdash C} (E\vee)$                        |

**Axiome :**

$$\frac{A \in \Gamma}{\Gamma \vdash A} (Ax)$$

***Tertium non datur* (logique classique) :**

$$\frac{}{\Gamma \vdash A \vee \neg A} (TND)$$

## A.2 Tableau : Calcul T

Start :

$$\frac{\Gamma, \neg A \vdash \perp}{\Gamma \vdash A} (S)$$

Axiomes :

$$\frac{\perp \in \Gamma}{\Gamma \vdash \perp} (Ax F) \quad \frac{A \in \Gamma \quad \neg A \in \Gamma}{\Gamma \vdash \perp} (Ax C)$$

Négation :

$$\frac{\neg \neg A \in \Gamma \quad \Gamma, A \vdash \perp}{\Gamma \vdash \perp} (N)$$

Règles pour opérateurs binaires :

|                   | Conjonctives ( $\alpha$ )                                                                                                        | Disjonctives ( $\beta$ )                                                                                                                            |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| $\wedge$          | $\frac{A \wedge B \in \Gamma \quad \Gamma, A, B \vdash \perp}{\Gamma \vdash \perp} (\alpha \wedge)$                              | $\frac{\neg(A \wedge B) \in \Gamma \quad \Gamma, \neg A \vdash \perp \quad \Gamma, \neg B \vdash \perp}{\Gamma \vdash \perp} (\beta \wedge)$        |
| $\vee$            | $\frac{\neg(A \vee B) \in \Gamma \quad \Gamma, \neg A, \neg B \vdash \perp}{\Gamma \vdash \perp} (\alpha \vee)$                  | $\frac{A \vee B \in \Gamma \quad \Gamma, A \vdash \perp \quad \Gamma, B \vdash \perp}{\Gamma \vdash \perp} (\beta \vee)$                            |
| $\longrightarrow$ | $\frac{\neg(A \longrightarrow B) \in \Gamma \quad \Gamma, A, \neg B \vdash \perp}{\Gamma \vdash \perp} (\alpha \longrightarrow)$ | $\frac{A \longrightarrow B \in \Gamma \quad \Gamma, \neg A \vdash \perp \quad \Gamma, B \vdash \perp}{\Gamma \vdash \perp} (\beta \longrightarrow)$ |

## A.3 Calcul des Tableaux : Propriétés et exemples

Le calcul de § A.2 est un calcul correct et complet pour la logique propositionnelle classique. Il procède essentiellement par simple décomposition de formules et a donc la propriété de la sous-formule : les formules générées par application d'une règle sont les sous-formules de formules existantes ; il n'est donc pas nécessaire d'inventer des formules.

Pour des opérateurs binaires, il y a deux grandes familles de règles :

- conjonctives ( $\alpha$ ) qui décomposent des formules essentiellement conjonctives, par exemple  $A \wedge B$  en  $A$  et  $B$  (règle  $(\alpha \wedge)$ ) pour créer un seul sous-but. Réécrire  $\neg(A \vee B)$  en  $\neg A \wedge \neg B$  et  $\neg(A \longrightarrow B)$  en  $A \wedge \neg B$  pour la justification de  $(\alpha \vee)$  et  $(\alpha \longrightarrow)$ .
- disjonctives ( $\beta$ ) qui décomposent des formules essentiellement disjonctives, par exemple  $A \vee B$  en  $A$  ou  $B$  (règle  $(\beta \vee)$ ) pour créer deux sous-buts.

Une preuve de  $\neg(p \longrightarrow q) \vdash p \vee q$  est :

$$\frac{\frac{\frac{\frac{\frac{\frac{\neg(p \longrightarrow q), \neg(p \wedge \neg q), p, \neg q, \neg p \vdash \perp}{\neg(p \longrightarrow q), \neg(p \wedge \neg q), p, \neg q \vdash \perp} (Ax C)}{\neg(p \longrightarrow q), \neg(p \wedge \neg q), p, \neg q, \neg q \vdash \perp} (Ax C)}{\neg(p \longrightarrow q), \neg(p \wedge \neg q), p, \neg q \vdash \perp} (\beta \wedge)}{\neg(p \longrightarrow q), \neg(p \wedge \neg q), p, \neg q \vdash \perp} (\alpha \longrightarrow)}{\neg(p \longrightarrow q), \neg(p \wedge \neg q) \vdash \perp} (S)}{\neg(p \longrightarrow q) \vdash p \wedge \neg q} (S)$$

La démarche typique est de rassembler toutes les formules à gauche de  $\vdash$ , en appliquant la règle  $(S)$ , puis d'appliquer les règles de décomposition  $((N), (\alpha), (\beta))$  jusqu'à trouver une contradiction avec  $(Ax F)$  ou  $(Ax C)$ .

## B Python

### B.1 Version typée de Python

Nous vous recommandons fortement d'utiliser une version typée de Python, où les paramètres d'une fonction et sa sortie sont annotés avec des types. Ces annotations sont une bonne documentation. Des warnings peuvent éliminer certaines erreurs de codage statiquement (avant exécution) et ainsi réduire drastiquement la phase de tests.

Une fonction qui s'écrit

---

```
def foo(x1, x2):
```

```
...
```

---

s'écrit :

---

```
def foo(x1 : T1, x2 : T2) -> TR:
```

```
...
```

---

avec les annotations, où les **T1**, **T2** sont les types des variables et **TR** le type de résultat. Les types peuvent être des types de base : **bool**, **int**, **str**, ou aussi **Form** pour notre type de formules, ou formés à partir de constructeurs de types, surtout :

- **tuple**[**T1**, **T2**] pour des paires de valeurs de types **T1**, **T2**
- **list**[**T**] pour une liste d'éléments de type **T**.
- **Callable**[**[T1, ..., Tn]**, **T**] le type des fonctions avec paramètres de type **T1**, ..., **Tn** et résultat **T**.

Une version banale de la division d'Euclide s'écrit donc :

---

```
def divEucl(x: int, y: int) -> tuple[int, int]:  
    return (x // y, x % y)
```

---

Vous savez que pour des raisons de décidabilité, les messages d'erreurs / warning (affichées par exemple par des éditeurs comme VSCode) ne sont pas toujours corrects et complets.

Si vous utilisez l'éditeur VSCode, nous vous recommandons d'installer le paquetage Pylance et d'activer le mode de vérification de types "strict". Pour le faire : ouvrez un fichier Python (par exemple **datastructs.py**), faites **Ctrl+**, (appuyer sur les touches "Ctrl" et "virgule" en même temps). Dans le dialogue qui s'ouvre, chercher le mot "typecheck". Ensuite, sélectionner le mode strict.

### B.2 Traitement de listes en Python

Vous pouvez utiliser librement des fonctions de haut niveau disponibles dans la librairie de Python<sup>7</sup>. Vous pouvez aussi utiliser des outils d'intelligence générative pour générer des bouts de code. Dans ce dernier cas, merci d'indiquer dans des commentaires quel outil vous avez utilisé, peut-être en affichant avec quel prompt vous avez obtenu la réponse.

Voici quelques schémas qui sont utiles surtout pour le traitement de listes, qui s'apparentent à des quantifications sur des ensembles ou de la compréhension d'ensembles :

- Compréhension de listes qui construit  $\{f(x) | x \in L \wedge P(x)\}$ .

Le résultat est une liste : **[f(x) for x in L if P(x)]**

*Exemple : tripler les éléments pairs d'une liste :*

---

```
>>> [3 * x for x in [1, 2, 3, 4, 5] if x % 2 == 0]  
[6, 12]
```

---

L'itération se fait aussi sur plusieurs listes. Utile pour former le produit cartésien de deux listes (et ensuite filtrer certains éléments) :

---

7. <https://docs.python.org/3/reference/index.html>

---

```
>>> [(x, y) for x in [1, 3, 5] for y in [2, 4, 6] if x < y]
[(1, 2), (1, 4), (1, 6), (3, 4), (3, 6), (5, 6)]
```

---

- Quantification existentielle :  $\exists x. x \in L \wedge P(x)$ .

Le résultat est un booléen : `any(P(x) for x in L)`.

*Exemple : est-ce que la liste contient des nombres divisibles par 3 ?*

---

```
>>> any(x % 3 == 0 for x in [1, 2, 3, 4, 5])
True
>>> any(x % 3 == 0 for x in [1, 2, 4, 5])
False
```

---

Similaire : quantificateur universel `all`.

### B.3 Fonctions avec des effets de bord en Python

Nous mettons en garde contre l'utilisation de fonctions ou méthodes avec effet de bord, qui peuvent avoir des effets collatéraux inattendus.

Voici une illustration :

---

```
def duplic(xs):
    return([xs, xs])

def add_to_first(xss: list[list[int]]):
    xs0 = xss[0]
    xs0.append(42)
    return(xss)
```

---

La fonction `duplic` renvoie une liste qui contient une liste donnée deux fois, tandis que `add_to_first` est censée rajouter 42 à la première sous-liste passée comme argument. Exécutez `add_to_first(duplic([1, 2]))` pour voir ce qui se passe réellement.

En général, il est conseillé d'éviter des fonctions qui font des mises à jour destructrices, comme `append`. Préférez des fonctions plus sûres comme la concaténation de deux listes, par exemple `xs0 + [42]`.